

Kontrolki natywne w .NET MAUI

Bartosz Miąskowski

Gdy standardowe kontrolki nie wystarczają

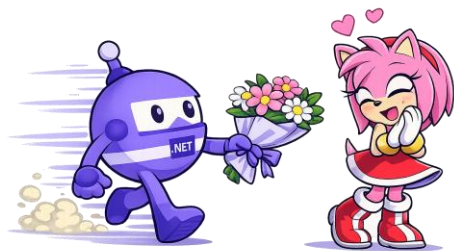
- .NET MAUI udostępnia zestaw **kontrolki** potrzebnych do tworzenia większości interfejsów aplikacji. Czasami jednak potrzebujemy czegoś więcej:
 - kontrolki, której .NET MAUI w ogóle nie posiada,
 - własnego, wielokrotnie używanego elementu interfejsu,
 - dostępu do kontrolki dostępnej natywnie tylko na konkretnej platformie.



Kontrolki dostępne domyślnie w MAUI **muszą być dostępne na każdej platformie**, więc brakuje wielu kontrolki specyficznych dla platformy – *przykładowo* GroupBox (Windows)

Jak rozszerzyć .NET MAUI?

- Gdy brakuje nam odpowiedniej kontrolki, mamy kilka możliwości:
 - stworzyć własną kontrolkę z istniejących elementów MAUI,
 - wydzielić kontrolki do własnej biblioteki i ponownie z nich korzystać w kolejnych projektach,
 - wykorzystać kontrolkę natywną dla konkretnej platformy.



Tworzenie własnej kontrolki

- Chcemy stworzyć **swoją własną kontrolkę** – przypominającą GroupBox z Windows Forms, która:
 - posiada **obramowanie**,
 - wyświetla **nagłówek na górnej krawędzi**,
 - pozwala umieścić **wewnątrz dowolną zawartość**,
 - może być **używana wielokrotnie** jak zwykła kontrolka .NET MAUI.
- Nie chcemy każdorazowo tworzyć kombinacji kontrolek i odpowiednich layoutów celem uzyskania oczekiwanego efektu – **chcemy całość zdefiniować w jednej kontrolce** i później **używać jej dokładnie tak samo jak zwykłe kontrolki** MAUI – jak <Button /> czy <Entry />.

```
<!-- Przykładowe użycie własnej kontrolki -->
<controls:GroupBox Header="Dane użytkownika"
  <VerticalStackLayout>
    ...
  </VerticalStackLayout>
</controls:GroupBox>
```

Z własnej kontrolki będziemy mogli korzystać jak z każdej innej w MAUI



Tworzenie własnej kontrolki – ContentView

- ContentView to kontrolka przeznaczona do tworzenia własnych, wielokrotnie używanych elementów interfejsu. Może przechowywać jeden element View w swojej właściwości Content. Własne kontrolki będą dziedziczyły po klasie (kontrolce) ContentView.

```
<ContentView x:Class="MauiApp.Controls.GroupBox">
</ContentView>
```

Kod kontrolki w XAML

```
public partial class GroupBox : ContentView
{
    public GroupBox()
    {
        InitializeComponent();
    }

    // np. HeaderProperty
}
```

Kod kontrolki w C#



Uwaga!

Wewnątrz <ContentView /> możemy umieścić wyłącznie jeden element.

Dobłą praktyką jest **umieszczanie własnych kontrol w katalogu Controls/** projektu.

Właściwości własnej kontrolki

- Większość kontroltek **aby była użyteczna musi posiadać własne właściwości**. W przypadku kontrolki typu `<GroupBox />` może być to właściwość `Header` – czyli tekst, który będzie wyświetlał się na górnej krawędzi obramowania.
- Właściwość może **być zwykłą właściwością w kodzie C#**, jednakże w ten sposób zdefiniowana **nie będzie mogła zostać użyta w kodzie XAML**.

```
public partial class GroupBox : ContentView
{
    public string Header { get; set; } // <-- Nasza właściwość

    public GroupBox()
    {
        InitializeComponent();
    }

    // np. HeaderProperty
}
```

Uwaga!

Do tak zdefiniowanej właściwości możemy się odwołać **wyłącznie z kodu C#**.

Właściwość w tym kształcie jest bezużyteczna w kontekście kontrolki XAML



Właściwości własnej kontrolki – BindableProperty

- Właściwości w własnej kontrolce nabierają więcej sensu, kiedy zaimplementujemy je przy pomocy klasy `BindableProperty`. Dzięki temu właściwość kontrolki będzie obsługiwała binding.



Ważne!

Wartość właściwości jest przechowywana w obiekcie `BindableProperty`, dlatego przy jej odczycie i zapisie wykonujemy rzutowanie obiektów.

```
public partial class GroupBox : ContentView
{
    public static readonly BindableProperty HeaderProperty =
        BindableProperty.Create( // <--- Tworzymy identyfikator właściwości
            nameof(Header), // <--- Nazwa właściwości
            typeof(string), // <--- Typ właściwości
            typeof(GroupBox), // <--- Klasa (kontrolka), do której należy właściwość
            string.Empty); // <--- Wartość domyślna

    public string Header
    {
        get => (string)GetValue(HeaderProperty);
        set => SetValue(HeaderProperty, value);
    }

    public GroupBox()
    {
        InitializeComponent();
    }
}
```

Konwencja – [nazwa_właściwości] + Property

Rzutowanie wartości właściwości

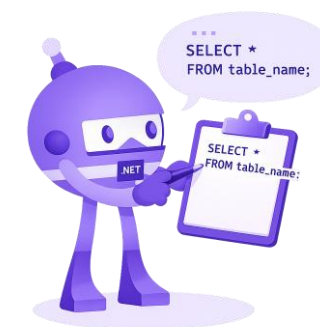
Poprawnie zaimplementowana właściwość w kontrolce MAUI

Właściwości własnej kontrolki – BindableProperty

- Po poprawnym zaimplementowaniu właściwości w kontrolce, przy użyciu obiektu BindableProperty, możemy przypisać właściwości wartość bezpośrednio w kodzie XAML oraz bez przeszkód używać bindingu.

```
<controls:GroupBox Header="Dane użytkownika" />
<controls:GroupBox Header="{Binding GroupTitle}" />
```

Przypisanie wartości do właściwości



Właściwość wewnątrz kontrolki

- Stworzyliśmy już pierwszą właściwość w naszej nowej kontrolce, teraz nadszedł czas aby z niej skorzystać.



Ważne!

Wewnątrz `<ContentView />` **najlepiej umieścić jakiś układ (layout)** – dokładnie tak samo, jak robi się to w przypadku strony – `<ContentPage />`.

Nazwa naszej kontrolki, niezbędne do przypisania właściwego BindingContextu

```
<ContentView x:Class="MauiApp.Controls.GroupBox" x:Name="groupBox">
  <!-- Pamiętaj, że ContentView może mieć tylko jeden element wewnątrz -->
  <Label Text="{Binding Header, Source={x:Reference groupBox}}" />
</ContentView>
```

Ustawiamy BindingContext tak, aby był on wewnątrz kontrolki, a nie strony

```
<!-- Alternatywna, prostsza wersja dla .NET 10 -->
<Label Text="{TemplateBinding Header}" />
```

Wersja alternatywna przekazania wartości do elementów zagnieżdżonych

Jak zbudować wygląd kontrolki?

- Kontrolka, którą budujemy będzie mogła **przechowywać w sobie inne kontrolki** – podobnie jak to jest z układami (*layoutami*) czy przykładowo `<Border />`.

```
<controls:GroupBox Header="Dane użytkownika">

    <VerticalStackLayout>
        <Entry Placeholder="Imię" />
        <Entry Placeholder="Nazwisko" />
    </VerticalStackLayout>

</controls:GroupBox>
```

Chcemy móc zagnieżdżać w naszej kontrolce inne elementy XAML – tak, jak jest to w klasycznym GroupBoxie.



Struktura wizualna kontrolki

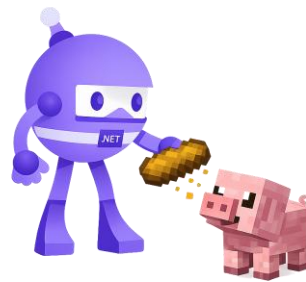
- Element `<ContentPresenter />` określa w którym miejscu pojawi się zawartość przekazana do kontrolki.

```
<ContentView>
  <Border Stroke="Gray"
          StrokeThickness="1"
          Padding="12">
    <ContentPresenter />
  </Border>
</ContentView>
```

Poprawnie zdefiniowana kontrolka

```
<controls:GroupBox>
  <VerticalStackLayout>
    <Label Text="Hello" />
    <Button Text="OK" />
  </VerticalStackLayout>
</controls:GroupBox>
```

Przekazanie zawartości do kontrolki



Dodatkowe właściwości – kolor tekstu

```
public static readonly BindableProperty HeaderTextColorProperty =  
    BindableProperty.Create(  
        nameof(HeaderTextColor),  
        typeof(Color),  
        typeof(GroupBox),  
        Colors.Black);  
  
public Color HeaderTextColor  
{  
    get => (Color)GetValue(HeaderTextColorProperty);  
    set => SetValue(HeaderTextColorProperty, value);  
}
```

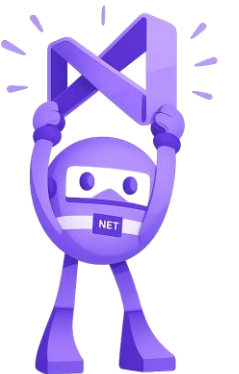
Właściwość definiująca kolor tekstu wewnątrz nowej kontrolki



Dodatkowe właściwości – grubość obramowania

```
public static readonly BindableProperty BorderThicknessProperty =  
    BindableProperty.Create(  
        nameof(BorderThickness),  
        typeof(double),  
        typeof(GroupBox),  
        1.0);  
  
public double BorderThickness  
{  
    get => (double)GetValue(BorderThicknessProperty);  
    set => SetValue(BorderThicknessProperty, value);  
}
```

Właściwość definiująca grubość obramowania



Dodatkowe właściwości

- Po zdefiniowaniu właściwości określających kolor tekstu i grubość obramowania, możemy bez problemu przekazać ich wartości do innych elementów wewnątrz kontrolki.

```
<Label
    Text="{Binding Header, Source={RelativeSource TemplatedParent}}"
    TextColor="{Binding HeaderTextColor, Source={RelativeSource TemplatedParent}}" />
<Border
    Stroke="{Binding BorderBrush, Source={RelativeSource TemplatedParent}}"
    StrokeThickness="{Binding BorderThickness, Source={RelativeSource TemplatedParent}}">
    <ContentPresenter />
</Border>
```

Przekazanie wartości właściwości do elementów zagnieżdżonych

```
<controls:GroupBox
    Header="Dane użytkownika"
    HeaderTextColor="DarkViolet"
    BorderBrush="DarkViolet"
    BorderThickness="2">
    <VerticalStackLayout>
        ...
    </VerticalStackLayout>
</controls:GroupBox>
```

Przekazanie wartości do właściwości własnej kontrolki



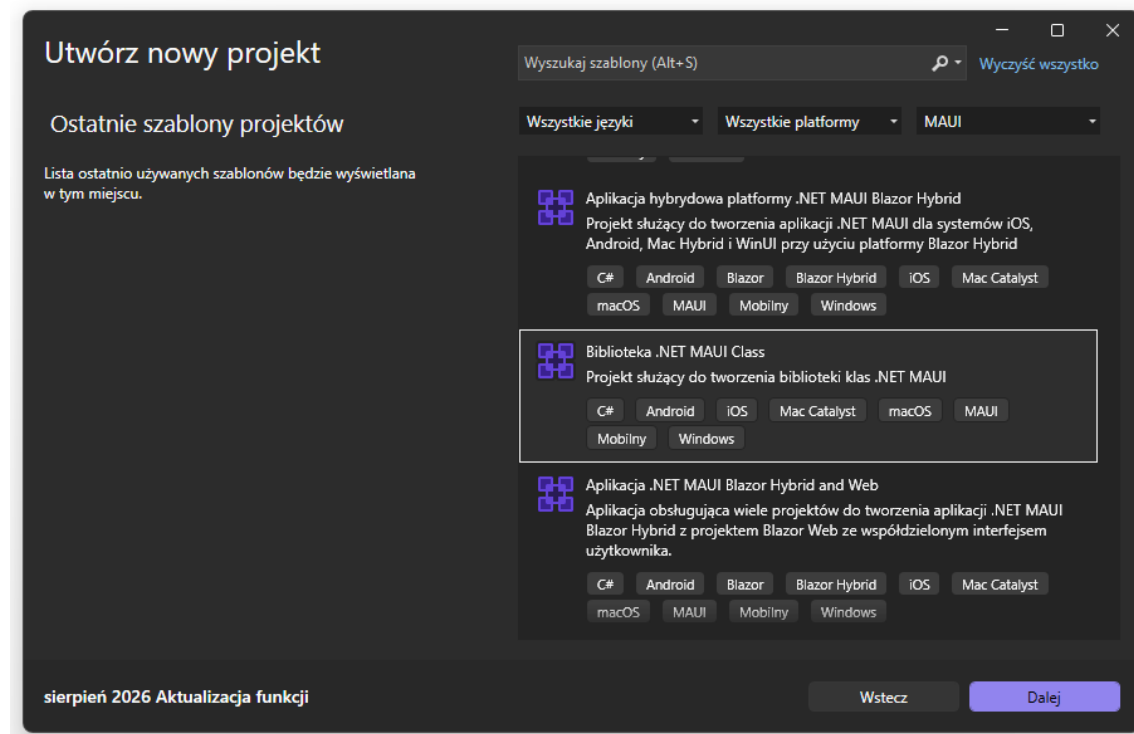
Wydzielanie własnych kontrollek

- Stworzyliśmy własną kontrolkę, jednakże **jest ona dostępna wyłącznie w obrębie danego projektu**. Ponowne jej wykorzystanie w innych projektach wymagałoby **nienieleganckiego kopiowania** kodu między projektami.
- W razie chęci naniesienia poprawek do kontrolki, kod musielibyśmy zmieniać w kilku projektach – ta sama bezsensowna praca.
- Możemy również podejść do tego inaczej, i **wydzielić kontrolkę do osobnej biblioteki**. Dzięki takiemu zabiegowi, kontrolka będzie mogła być **łatwo ponownie użyta przez kolejne projekty MAUI**.



Biblioteka klas .NET MAUI

- Do wydzielenia własnych kontroltek możemy skorzystać z Biblioteki klas .NET MAUI – działa to analogicznie jak w przypadku zwykłych bibliotek C#, z tą różnicą, że w bibliotece klas .NET MAUI możemy również umieszczać kod zależny od frameworka .NET MAUI – jak chociażby własne kontrolki.



Wybór projektu biblioteki klas .NET MAUI w Visual Studio

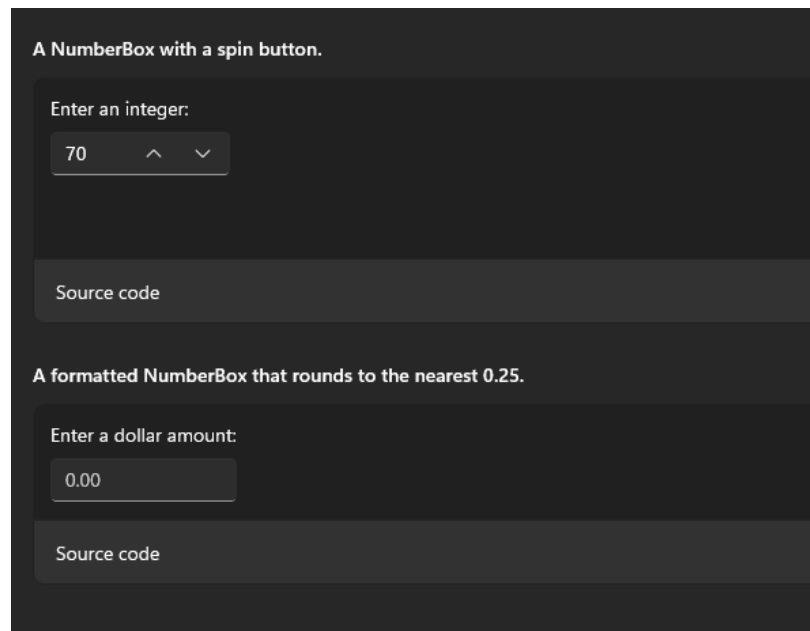


Kontrolki natywne

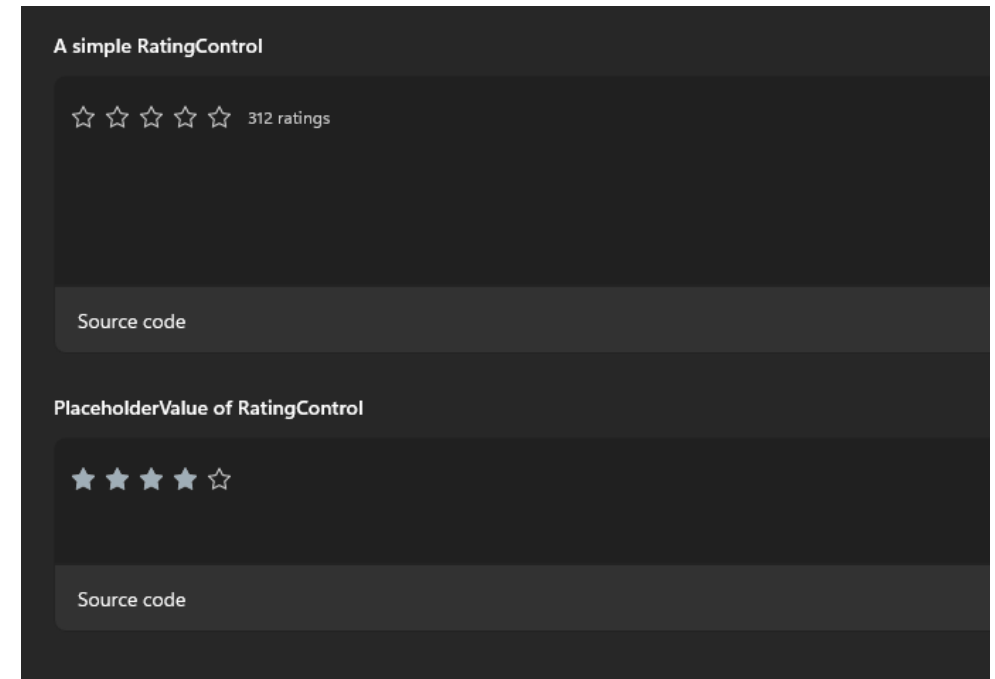
- .NET MAUI daje nam dostęp do zbioru kontroltek **dostępnych na wszystkich platformach**:
 - <Button />, <Entry />, <Label />, <Slider />, <Switch /> itd..
- Poszczególne platformy mają też własne kontrolki, których MAUI nie udostępnia bezpośrednio:
 - Windows – WinUI 3,
 - Android,
 - macOS, iOS – Mac Catalyst.



Brakujące kontrolki – WinUI 3



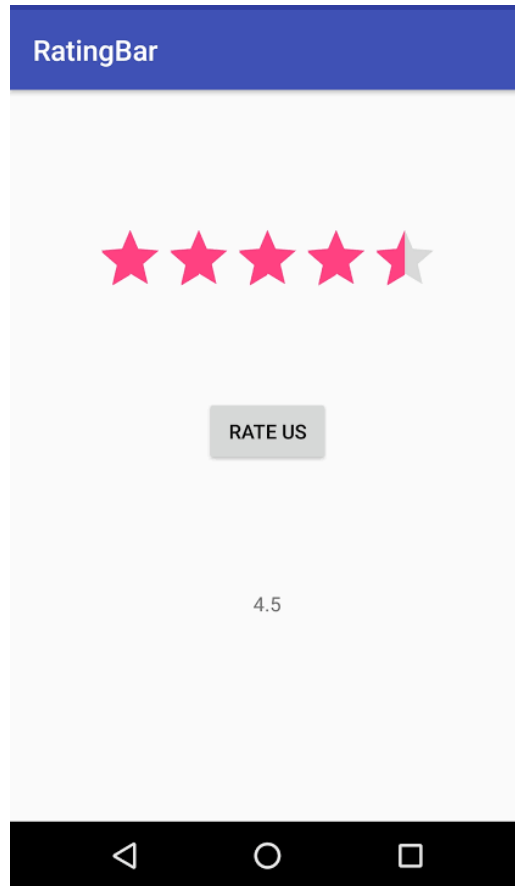
NumberBox



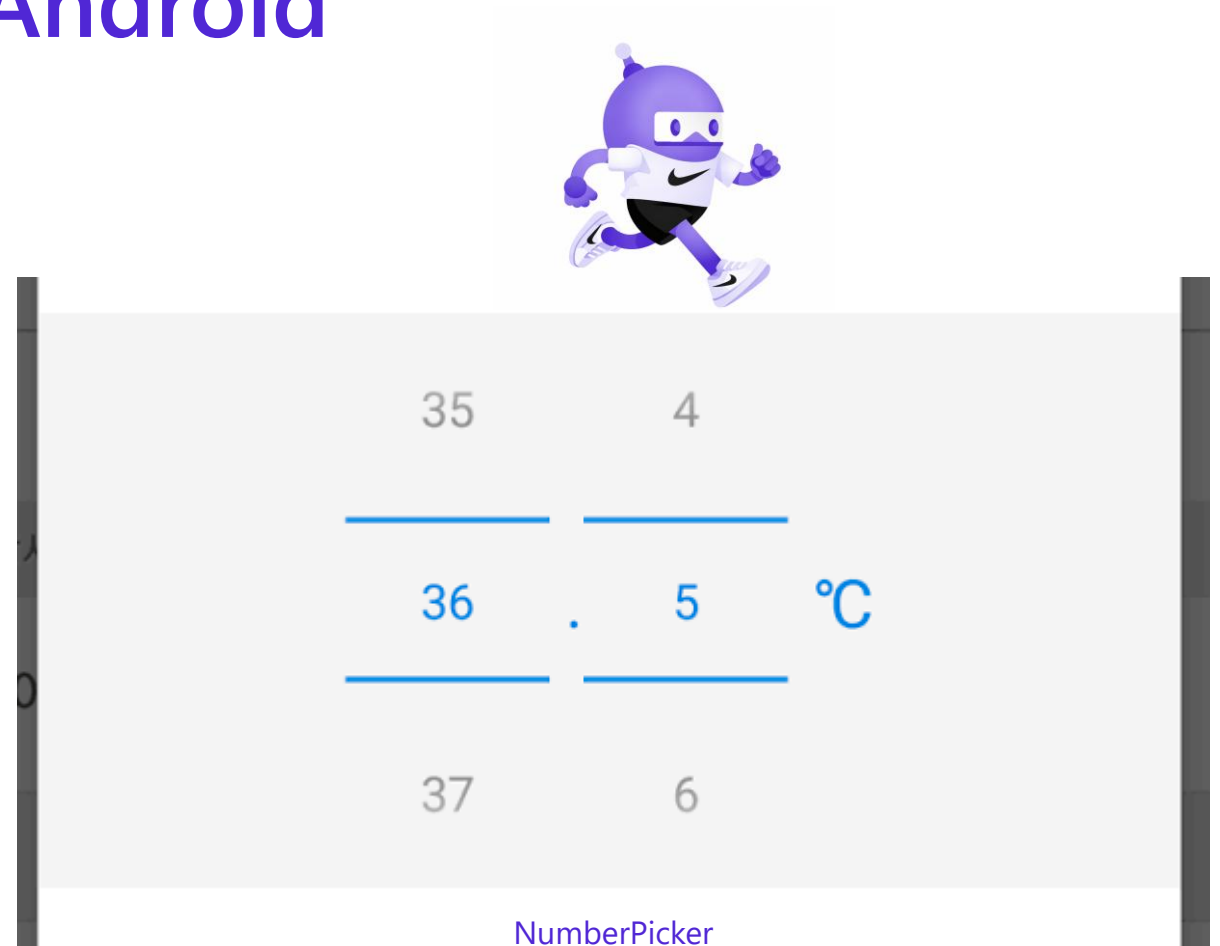
RatingControl



Brakujące kontrolki – Android

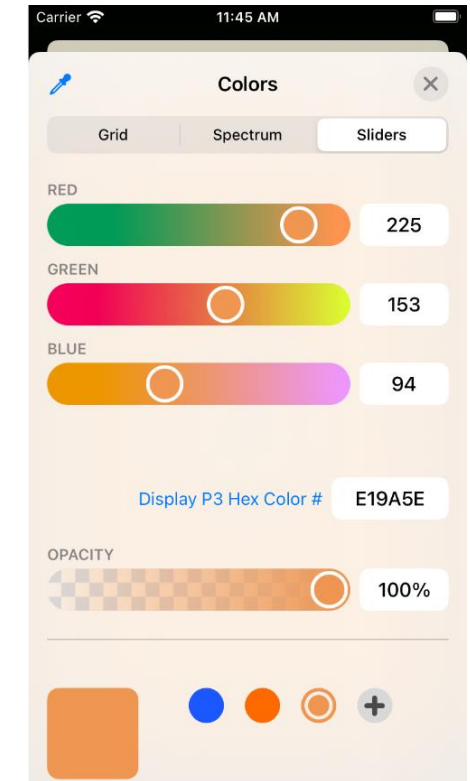
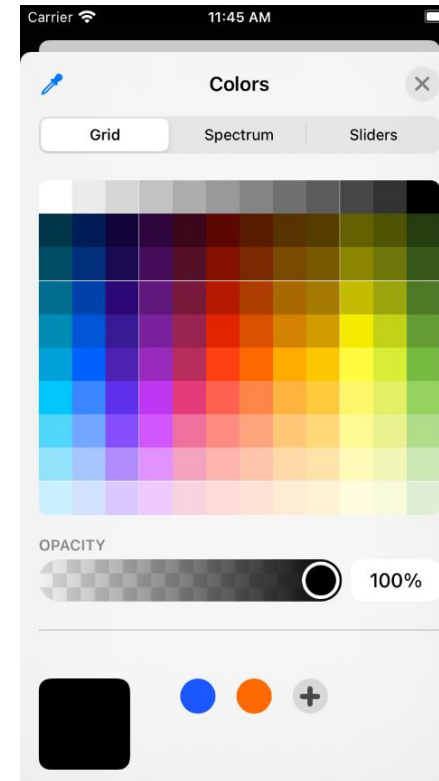
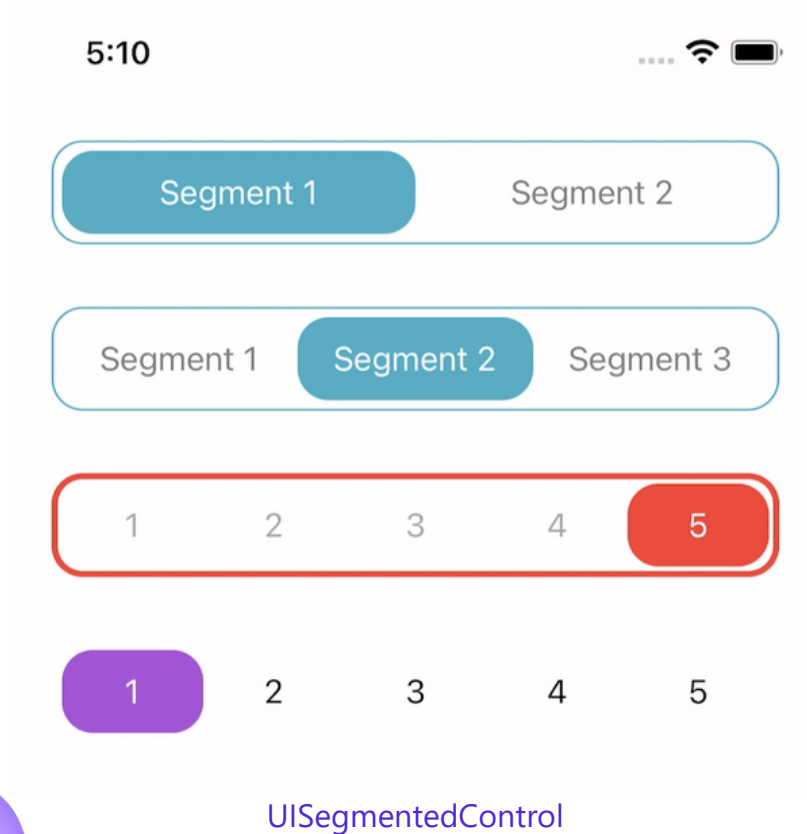


RatingBar



NumberPicker

Brakujące kontrolki – Mac Catalyst

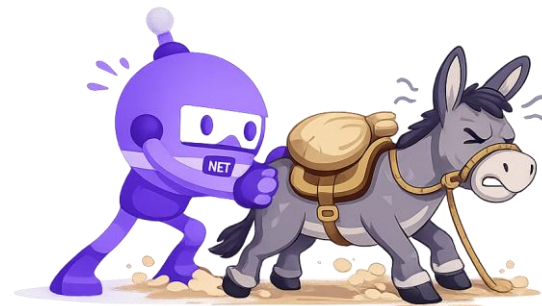


UIColorWell

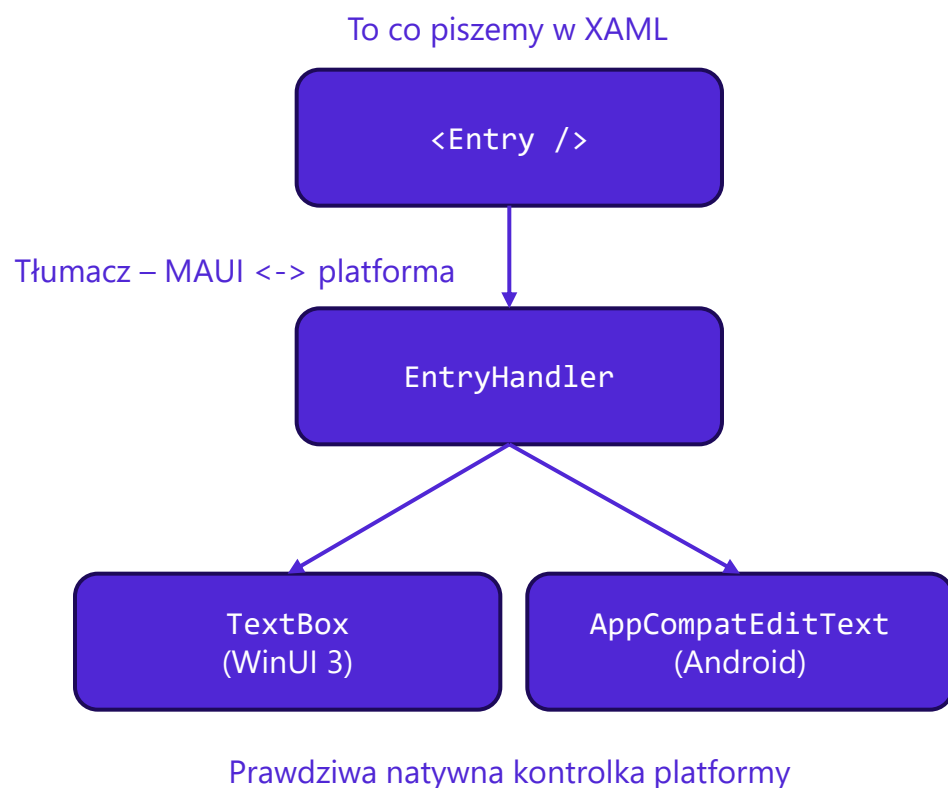


Używanie natywnych kontrolek

- Fakt, że .NET MAUI nie posiada wbudowanej obsługi danej kontrolki, **nie oznacza, że nie możemy jej użyć**.
- Dodanie do aplikacji natywnej kontrolki sprawi, że będzie ona dostępna **wyłącznie w wersji skompilowanej na platformę**, z której pochodzi kontrolka.
- Korzystanie z natywnych kontrolek **zwiększa wachlarz możliwości** aplikacji, ale znacząco **utrudnia jej utrzymanie** na wielu platformach.



Jak MAUI tworzy kontrolki?



- Każda kontrolka MAUI jest tłumaczona na natywną wersję dla danej platformy (to już doskonale wiemy z IV klasy).
- Przy obsłudze natywnych kontrolerek, przyda nam się kilka pojęć:
 - **VirtualView** – kontrolka MAUI, wspólna dla wszystkich platform (np. *Entry*),
 - **Handler** – klasa, która tworzy natywną kontrolkę i przekazuje jej wartości (np. *EntryHandler*),
 - **PlatformView** – natywna kontrolka konkretnej platformy (np. *TextBox*, *AppCompatActivity*).
- Handler ma dwa zadania:
 - Tłumaczenie z MAUI do kodu platformy – gdy zmienia się *Entry.Text*, handler ustawia *TextBox.Text*.
 - Tłumaczenie z kodu platformy do MAUI – gdy użytkownik wpisze coś w *TextBox*, handler aktualizuje *Entry.Text*.

Kontrolka NumberBox – WinUI 3

- Chcielibyśmy w widoku MAUI skorzystać z kontrolki NumberBox, [dostępnej w WinUI 3](#). Służy ona do wprowadzania liczb:
 - jest polem tekstowym, które przyjmuje [tylko wartości liczbowe](#),
 - posiada strzałki w górę i w dół – do inkrementacji i dekrementacji wprowadzanej wartości,
 - posiada zakres [minimum i maximum](#) dla wprowadzanych wartości,
 - umożliwia ustawienie kroku zmiany wartości – step,
 - pozwala na łatwe [formatowanie liczb](#),
 - potrafi policzyć proste wyrażenia np. $2*8$ zwróci 16, jeżeli ustawimy właściwość `AcceptsExpression`.

```
<!-- Deklaracja NumberBox w WinUI 3 -->
<NumberBox Minimum="0"
            Maximum="100"
            Value="20"
            SmallChange="5"
            SpinButtonPlacementMode="Inline" />
```

Tak wyglądałby XAML w projekcie WinUI 3 – nie zadziała on w .NET MAUI



Kontrolka NumberBox – WinUI 3

- Chcemy do projektu dodać obsługę kontrolki NumberBox (Win UI) oraz NumberPicker (Android). W tym celu [wykorzystamy katalog Platforms wewnątrz projektu](#), celem umieszczenia tam logiki odpowiedzialnej za obsługę każdej z tych natywnych kontrolek.

Struktura projektu:

- Controls/
 - NumberBoxView.cs (*własna kontrolka – MAUI*)
- Handlers/
 - NumberBoxViewHandler.cs (*część wspólna handlera*),
- Platforms/
 - Windows/
 - NumberBoxViewHandler.cs (*część tylko dla Windows*),
 - Android
 - NumberBoxViewHandler.cs (*tylko dla dla Androida*)

Ważne!

Klasa NumberBoxViewHandler to jedna klasa w trzech plikach – niezbędne jest słowo kluczowe **partial**. Część wspólna mówi co ma być mapowane, części platformowe – jak.



Kod kontrolki

- Zaczniemy od stworzenia pliku w katalogu Controls/ (tak, jak w przypadku własnej kontrolki), gdzie zdefiniujemy m.in. właściwości.

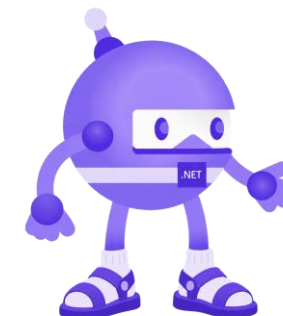
```
public class NumberBoxView : View
{
    public static readonly BindableProperty ValueProperty =
        BindableProperty.Create(
            nameof(Value),
            typeof(double),
            typeof(NumberBoxView),
            0d,
            defaultBindingMode: BindingMode.TwoWay);

    public double Value
    {
        get => (double)GetValue(ValueProperty);
        set => SetValue(ValueProperty, value);
    }
}
```

Obsługa ustawiania i odczytywania wartości (obustronna)

Uwaga!

W tym przypadku klasa naszej kontrolki dziedziczy po View a nie ContentView dlatego, że kontrolka nie będzie miała zawartości MAUI – będzie to natywna kontrolka.



Kod kontrolki – dodatkowe właściwości

- Kontrolka w zależności od platformy, **będzie się różniła dostępnymi właściwościami**. Niemniej, możemy dodać obsługę tych najważniejszych – jeżeli czegoś brakuje na danej platformie, **handler zignoruje tę właściwość**.

```
public static readonly BindableProperty MinimumProperty =  
    BindableProperty.Create(  
        nameof(Minimum), typeof(double), typeof(NumberBoxView), 0d);  
  
public static readonly BindableProperty MaximumProperty =  
    BindableProperty.Create(  
        nameof(Maximum), typeof(double), typeof(NumberBoxView), 100d);  
  
public static readonly BindableProperty SmallChangeProperty =  
    BindableProperty.Create(  
        nameof(SmallChange), typeof(double), typeof(NumberBoxView), 1d);  
  
public double Minimum  
{  
    get => (double)GetValue(MinimumProperty);  
    set => SetValue(MinimumProperty, value);  
}  
  
public double Maximum  
{  
    get => (double)GetValue(MaximumProperty);  
    set => SetValue(MaximumProperty, value);  
}  
  
public double SmallChange  
{  
    get => (double)GetValue(SmallChangeProperty);  
    set => SetValue(SmallChangeProperty, value);  
}
```

Dodajemy obsługę Minimum, Maximum i SmallChange (tej nie będzie na Androidzie)



Wspólny handler

- Klasa NumberBoxViewHandler dziedziczy po ViewHandler<TView, TPlatformView>, w związku z tym już w tym miejscu będziemy musieli skorzystać z dyrektywy preprocesora, która **uzależni importowane przestrzenie nazw od celu kompilacji**.

```
#if WINDOWS
using PlatformView = Microsoft.UI.Xaml.Controls.NumberBox;
#elif ANDROID
using PlatformView = Android.Widget.NumberPicker;
#endif

namespace NativeControls.Handlers;

public partial class NumberBoxViewHandler : ViewHandler<NumberBoxView, PlatformView>
{
}
```

Szkielet klasy handlera – importowane przestrzenie nazw zależą od platformy docelowej



Wspólny handler – mapowanie właściwości

- W wspólnej części handlera umieścimy **mapowanie właściwości** – tych, które zadeklarowaliśmy w kodzie kontrolki.

```
public partial class NumberBoxViewHandler : ViewHandler<NumberBoxView, PlatformView>
{
    public static readonly IPropertyMapper<NumberBoxView, NumberBoxViewHandler> PropertyMapper =
        new PropertyMapper<NumberBoxView, NumberBoxViewHandler>(ViewMapper)
        {
            [nameof(NumberBoxView.Value)] = MapValue,
            [nameof(NumberBoxView.Minimum)] = MapMinimum,
            [nameof(NumberBoxView.Maximum)] = MapMaximum,
            [nameof(NumberBoxView.SmallChange)] = MapSmallChange
        };

    public NumberBoxViewHandler() : base(PropertyMapper)
    {
    }
}
```

Widoczne delegaty utworzymy dopiero w kodzie dla danej platformy.

Mapowanie właściwości odbywa się przez stworzenie słownika **nazwa właściwości – delegata**.



Handler natywnej kontrolki – WinUI 3

```
using Microsoft.UI.Xaml.Controls;  
using NativeControls.Controls;  
  
namespace NativeControls.Handlers;  
  
public partial class NumberBoxViewHandler  
{  
    protected override NumberBox CreatePlatformView()  
    {  
        return new NumberBox  
        {  
            // Właściwości, dostępne w natywnej kontrolce  
            SpinButtonPlacementMode = NumberBoxSpinButtonPlacementMode.Inline  
        };  
    }  
  
    private static void MapValue(NumberBoxViewHandler handler, NumberBoxView view)  
    => handler.PlatformView.Value = view.Value;  
  
    private static void MapMinimum(NumberBoxViewHandler handler, NumberBoxView view)  
    => handler.PlatformView.Minimum = view.Minimum;  
  
    private static void MapMaximum(NumberBoxViewHandler handler, NumberBoxView view)  
    => handler.PlatformView.Maximum = view.Maximum;  
  
    private static void MapSmallChange(NumberBoxViewHandler handler, NumberBoxView view)  
    => handler.PlatformView.SmallChange = view.SmallChange;  
}
```

Natywne właściwości danej kontrolki

Handler dla natywnej kontrolki WinUI 3 – wraz z metodami mapującymi

Uwaga!

Kod w katalogu danej platformy w Platforms/ **nie** wymaga dyrektyw kompilacji warunkowej.



Handler natywnej kontrolki – WinUI 3

- Do tej pory zmapowaliśmy jedynie właściwości natywnej kontrolki, potrzebujemy jeszcze obsługi zdarzeń.

```
public partial class NumberBoxViewHandler
{
    protected override void ConnectHandler(NumberBox platformView)
    {
        base.ConnectHandler(platformView);
        platformView.ValueChanged += OnValueChanged;

        // Możemy również dodać obsługę innych zdarzeń w natywnej kontrolce
    }

    protected override void DisconnectHandler(NumberBox platformView)
    {
        platformView.ValueChanged -= OnValueChanged; // <-- Bardzo ważne, ponieważ?
        base.DisconnectHandler(platformView);
    }

    private void OnValueChanged(NumberBox sender, NumberBoxValueChangedEventArgs args)
    {
        if (VirtualView is null)
            return;

        if (double.IsNaN(args.NewValue))
            return;

        VirtualView.Value = args.NewValue;
    }
}
```

Dzięki temu będziemy mogli zbindować wartość kontrolki z właściwością VM

Obsługa zdarzenia zmiany wartości w platformowym handlerze



Rejestracja handlerów

- Aby aplikacja wiedziała, z jakiego handlera ma skorzystać w przypadku danej kontrolki, musimy całość zarejestrować w `MauiProgram.cs` – podobnie jak robimy to z VM.

```
using NativeControls.Controls;
using NativeControls.Handlers;

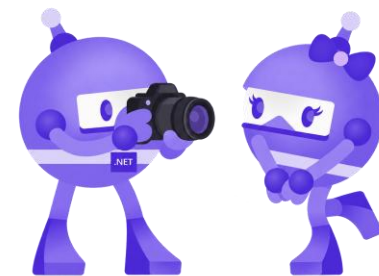
public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();

        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
            })

            // Rejestrowanie naszych handlerów
            .ConfigureMauiHandlers(handlers =>
            {
                handlers.AddHandler<NumberBoxView, NumberBoxViewHandler>();
            });

        return builder.Build();
    }
}
```

Rejestrowanie handlera dla naszej natywnej kontrolki



MVVM oraz natywna kontrolka

- Przy poprawnym stworzeniu własnej kontrolki (*obsłudze natywnej*) będziemy mogli skorzystać z MVVM tak samo *prosto*, jak w przypadku domyślnych kontroltek MAUI.

```
<!-- Pamiętaj o zaimportowaniu przestrzeni nazw z nową kontrolką -->
<ContentPage x:Class="NativeControls.MainPage"
  xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:controls="clr-namespace:NativeControls.Controls"
  xmlns:vm="clr-namespace:NativeControls.ViewModels"
  x:DataType="vm:MainViewModel">
  <VerticalStackLayout Padding="20" Spacing="10">
    <Label Text="Ilość:" />

    <controls:NumberBoxView Value="{Binding Amount}"
      Minimum="0"
      Maximum="100"
      SmallChange="5" />

    <Label Text="{Binding Amount, StringFormat='Wybrano: {0}'}" />
  </VerticalStackLayout>
</ContentPage>
```

Podłączenie VM do nowej kontrolki



Handler natywnej kontrolki – Android

```
using Android.Widget;
using NativeControls.Controls;

namespace NativeControls.Handlers;

public partial class NumberBoxViewHandler
{
    // Na Androidzie odpowiednik kontrolki NumberBox to NumberPicker
    protected override NumberPicker CreatePlatformView()
    {
        // Kontrolki Androidowe wymagają przekazania innych argumentów
        return new NumberPicker(Context) // Activity – slogan z Kotlin i Android Studio
        {
            WrapSelectorWheel = false
        };
    }
}
```

Handler kontrolki - Android



Handler natywnej kontrolki – Android

```
public partial class NumberBoxViewHandler
{
    private static void MapValue(NumberBoxViewHandler handler, NumberBoxView view)
        => UpdatePicker(handler.PlatformView, view);

    private static void MapMinimum(NumberBoxViewHandler handler, NumberBoxView view)
        => UpdatePicker(handler.PlatformView, view);

    private static void MapMaximum(NumberBoxViewHandler handler, NumberBoxView view)
        => UpdatePicker(handler.PlatformView, view);

    private static void MapSmallChange(NumberBoxViewHandler handler, NumberBoxView view)
    {
        // NumberPicker nie ma kroku - ignorujemy
    }

    private static void UpdatePicker(NumberPicker picker, NumberBoxView view)
    {
        // Odpowiednik Androidowy obsługuje liczby całkowite
        picker.MaxValue = (int)view.Maximum;
        picker.MinValue = (int)view.Minimum;
        picker.Value = (int)view.Value;
    }
}
```

Handler kontrolki – Android, mapowanie właściwości



Handler natywnej kontrolki – Android

```
public partial class NumberBoxViewHandler
{
    protected override void ConnectHandler(NumberPicker platformView)
    {
        base.ConnectHandler(platformView);
        platformView.ValueChanged += OnValueChanged;
    }

    protected override void DisconnectHandler(NumberPicker platformView)
    {
        platformView.ValueChanged -= OnValueChanged;
        base.DisconnectHandler(platformView);
    }

    private void OnValueChanged(object? sender, NumberPicker.ValueChangeEventArgs e)
    {
        if (VirtualView is null)
            return;

        VirtualView.Value = e.NewVal;
    }
}
```

Handler kontrolki – Android, obsługa zdarzeń



Pozostałe platformy

- Jeżeli tworzymy handler dla danej kontrolki, niezbędne jest **dodanie jego obsługi na każdej z docelowych platform aplikacji** – w przeciwnym razie, projekt się nie skompiluje.
- Jeżeli nie planujesz obsługi danej platformy, należy usunąć ją z kodu projektu aplikacji – **csproj**.

```
NativeControls.csproj  ⚙  ✕
1  <Project Sdk="Microsoft.NET.Sdk">
2
3      <PropertyGroup>
4          <TargetFrameworks>net10.0-android</TargetFrameworks>
5          <TargetFrameworks Condition="!$([MSBuild]::IsOSPlatform('linux'))">$(TargetFrameworks);net10.0-ios;net10.0-maccatalyst</TargetFrameworks>
6          <TargetFrameworks Condition="$([MSBuild]::IsOSPlatform('windows'))">$(TargetFrameworks);net10.0-windows10.0.19041.0</TargetFrameworks>
7
8          <!-- Note for MacCatalyst:
9              The default runtime is maccatalyst-x64, except in Release config, in which case the default is maccatalyst-x64;maccatalyst-arm64.
10             When specifying both architectures, use the plural <RuntimeIdentifiers> instead of the singular <RuntimeIdentifier>.
11             The Mac App Store will NOT accept apps with ONLY maccatalyst-arm64 indicated;
12             either BOTH runtimes must be indicated or ONLY macatalyst-x64. -->
13             <!-- For example: <RuntimeIdentifiers>maccatalyst-x64;maccatalyst-arm64</RuntimeIdentifiers> -->
14
```

Jeżeli nie planujemy obsługi danej platformy, po prostu usuwamy dany `<TargetFrameworks />`

